Course Outline: Choosing the Right Database

Title: Choosing the Right Database **Skill:** Skill 40: Cómo optimizar el almacenamiento para reporte, integridad, consulta o transporte **Learnpack:** + ¿Cómo escoger la base de datos adecuada para datos? **File:** s40-w14d40-como-escoger-la-base-de-datos-adecuada-p **Prerequisite:** Introducción a Data Pipelines **Target Audience:** Beginner developers who understand data pipelines and basic SQL **Total Lessons:** 11 **Format:** Conceptual (0% hands-on)

Course Description

Every application stores data — but most developers choose a database out of habit, not judgment. That default decision can cost your application its performance, its correctness, or both. This course teaches you to choose deliberately: understand the SQL vs NoSQL divide, learn how workload requirements (reports, integrity, read volume) drive the choice, and see how the major databases in the world stack up against each other.

Course Philosophy

This course emphasizes:

- Decision-making over memorization: the goal is a mental model, not a list of facts
- Workload-first thinking: what your app *does* determines what storage it *needs*
- Real-world grounding: every concept is tied to the kinds of systems you'll actually build

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The SQL vs NoSQL Divide	3
02 - Matching Databases to Workloads	5
03 - Assessment	1
04 - Outro	1
Total	**11**

00.0 Choosing the Right Database 🕮

Learning Objectives:

- Understand why database selection is a design decision with long-term consequences
- Preview the two sections and the decision framework you'll build in this course

Content Outline:

- The problem: most developers pick a database by default (PostgreSQL because I've seen it, MongoDB because someone recommended it)
- Why the default choice costs you: mismatched workloads cause slow queries, broken transactions, and wasted infrastructure
- What students will be able to do by the end: map any workload requirement to the right type of database and justify the choice
- Course overview: Section 01 gives you the conceptual map (SQL vs NoSQL), Section 02 gives you the practical selection tools (workload matching + the major databases)

Transitions: This course picks up where the data pipelines course left off. You now know how data flows — this course answers where to store it and why that matters.

Key Principles:

- No database is universally better — each one dominates specific use cases
- The "right" database is the one that matches your workload, not the one you know best

Examples:

- A startup uses MongoDB for everything — including financial records that need strict integrity. A bad transaction leaves an account balance inconsistent. A relational database with ACID guarantees would have caught it.
- A data team runs millions of analytical queries on a PostgreSQL instance shared with their production app. Queries are slow and the app suffers. BigQuery would have isolated the analytical load entirely.

01.0 The SQL vs NoSQL Divide 🕮

Learning Objectives:

- Understand what separates SQL (relational) databases from NoSQL (non-relational) databases
- Recognize the core trade-offs each model makes
- Know which sub-lessons will cover each model in depth

Content Outline:

- What all databases share: persistent storage, query interface, consistency guarantees (to varying degrees)
- The fundamental divide: how data is structured and how relationships are expressed
  - SQL: data lives in tables; relationships are enforced via foreign keys and joins
  - NoSQL: data lives in documents, key-value pairs, wide columns, or graphs; relationships are embedded or referenced loosely
- The trade-off in plain terms: SQL gives you structure and consistency; NoSQL gives you flexibility and scale
- Overview of Section 01: 01.1 covers SQL, 01.2 covers NoSQL

Transitions: This lesson frames the two models conceptually. Lessons 01.1 and 01.2 will each take one model in depth — what makes it strong, when it breaks down, and when to reach for it.

Key Principles:

- SQL databases enforce structure at write time — you define the schema before data goes in
- NoSQL databases enforce structure at read time — you shape the data when you retrieve it
- Neither model is inherently better; the right one depends on your data's shape and your app's access patterns

Examples:

- A user table in PostgreSQL: defined columns, types, constraints, foreign keys to an orders table
- A user document in MongoDB: a JSON blob with nested preferences, a list of past orders embedded inline — no joins needed, but no enforcement of field presence either

---

01.1 When SQL Fits 🕮

Learning Objectives:

- Identify the characteristics of data and workloads that make a relational database the right choice
- Understand what ACID means and why it matters
- Recognize when SQL is being misused

Content Outline:

- What SQL databases are built for:
  - Structured, tabular data with predictable schemas
  - Relationships that need to be enforced (not just expressed)
  - Workloads that require transactional consistency (ACID)
- ACID explained:
  - Atomicity: a transaction either completes fully or not at all
  - Consistency: every transaction brings the database from one valid state to another
  - Isolation: concurrent transactions don't interfere with each other
  - Durability: committed transactions survive crashes
- When SQL is the right tool:
  - Financial systems, e-commerce orders, booking systems — anywhere data integrity is a hard requirement
  - Relational data that benefits from joins (users ↔ orders ↔ products)
  - Reporting and aggregation on structured data (GROUP BY, window functions)
- When SQL struggles:
  - Unstructured or highly variable data (different fields per record)
  - Horizontal scale (sharding relational databases is complex and expensive)
  - Write-heavy workloads at massive scale
- Anti-pattern: using SQL to store semi-structured data (JSON blobs in TEXT columns) — you lose all the relational guarantees while adding parsing complexity

Transitions: SQL's strength is its guarantees. The next lesson covers NoSQL — a family of databases that trade some of those guarantees for flexibility and scale.

Key Principles:

- ACID guarantees are not free — they impose coordination costs that limit write throughput at scale
- "I know SQL" is not a reason to use SQL; workload fit is the reason

Examples:

- A bank transfer: debit one account, credit another. If anything fails mid-transaction, the whole operation rolls back. ACID makes this safe.
- An e-commerce order: customer record, line items, inventory decrement, payment record — all written atomically or not at all.

01.2 When NoSQL Fits 🕮

Learning Objectives:

- Identify the data shapes and access patterns that favor NoSQL databases
- Distinguish between the main NoSQL families and what each is optimized for
- Recognize when NoSQL is being misused

Content Outline:

- Why NoSQL exists: web-scale applications needed horizontal scaling and flexible schemas that relational databases couldn't easily provide
- The four main NoSQL families:
  - **Document stores** (MongoDB, DynamoDB): store JSON-like documents; best for objects with variable fields and nested structures
  - **Key-value stores** (Redis, DynamoDB): extremely fast lookups by key; best for caching, sessions, leaderboards
  - **Wide-column stores** (Cassandra, BigTable): store data in column families; best for massive write throughput and time-series data
  - **Graph databases** (Neo4j): store nodes and edges; best for relationship-heavy queries (social networks, fraud detection)
- When NoSQL is the right tool:
  - Variable or evolving schemas (user-generated content, product catalogs with different attributes per product)
  - High write throughput at scale (event logging, clickstreams, IoT telemetry)
  - Data that maps naturally to a document (a blog post with tags, author, content — all in one place)
  - Caching hot data for microsecond reads (Redis)
- When NoSQL struggles:
  - Complex multi-entity transactions (no ACID across collections)
  - Ad-hoc queries with complex joins (documents don't join — you denormalize instead, and denormalization costs you at update time)

    - Data with strict structural requirements where invalid data must be rejected at write time
- Anti-pattern: using MongoDB because "JSON is easier" for highly relational data — you end up manually enforcing referential integrity that a relational database would have given you for free

Transitions: Now that you understand both models, the next lesson uses them together: how do you draw the line between them for a real project?

Key Principles:

- NoSQL databases trade consistency guarantees for scalability and flexibility — that trade-off is sometimes worth it and sometimes not
- "Flexible schema" is a feature when your data genuinely varies; it's a liability when it lets invalid data in

Examples:

- A product catalog where electronics have wattage and warranty fields, clothing has size and material fields — MongoDB handles variable schemas naturally
- A gaming leaderboard updated thousands of times per second — Redis sorted sets are purpose-built for this; a relational database would be a bottleneck

---

02.0 Matching Databases to Workloads 🕮

Learning Objectives:

- Understand what "workload" means in the context of database selection
- Know the three main workload categories this section covers and how they drive different database choices
- See how Section 02 extends the SQL vs NoSQL conceptual framework into practical selection

Content Outline:

- What a workload is: the type of work an application asks its database to do — how data is written, read, queried, and transformed
- The three workload dimensions this section covers:
    1. **Orientation to reports**: the database serves analytical queries over large datasets
    2. **Integrity as a requirement**: the database must guarantee correctness across multi-step operations
    3. **Read-heavy traffic**: the database serves far more reads than writes, often at high concurrency
- How the same data might call for different databases depending on how it's used
- Overview of 02.1, 02.2, 02.3, 02.4

Transitions: Lessons 01.1 and 01.2 gave you the models. This section translates them into decisions: given what your application *does*, what database does it *need*?

Key Principles:

- Workload drives storage — understand the dominant access pattern before choosing a database

- An application can have multiple workloads (e.g., writes go to PostgreSQL, analytical queries go to BigQuery, hot lookups go to Redis)

Examples:

- A social media app: user profiles and posts stored in a document database (variable structure, fast writes) — but engagement analytics run on a data warehouse (large-scale aggregation)
- A payment processor: transactions stored in a relational database (ACID guarantees) — but fraud detection models read from a high-speed key-value store (sub-millisecond lookups)

02.1 Report-Oriented Workloads 🕮

Learning Objectives:

- Understand what makes a workload "report-oriented" and why OLTP databases struggle with it
- Identify databases optimized for analytical queries
- Know when to separate your analytical storage from your operational storage

Content Outline:

- OLTP vs OLAP:
  - **OLTP** (Online Transaction Processing): optimized for fast, individual reads/writes — typical app databases (PostgreSQL, MySQL, MongoDB)
  - **OLAP** (Online Analytical Processing): optimized for scanning large volumes of data and computing aggregations — data warehouses
- Why OLTP databases struggle with analytical queries:
  - Row-oriented storage reads entire rows even when only a few columns are needed
  - Large scans compete with live application traffic
  - Indexes optimized for point lookups don't help full-table aggregations
- Column-oriented storage: how data warehouses store data column-by-column to make aggregation queries extremely fast
- Databases optimized for reports:
  - **BigQuery**: serverless, columnar, SQL-based; scales to petabytes; billed per query
  - **Redshift**: Amazon's columnar data warehouse; good for existing AWS stacks
  - **Snowflake**: cloud-agnostic data warehouse; separates compute from storage
- When to build a separate analytical layer:
  - When analytical queries slow down your production database
  - When your data exceeds what a single relational instance can aggregate efficiently
  - When you need to query data from multiple sources in one place (ETL pipelines feed the warehouse)
- Anti-pattern: running analytics directly on a production OLTP database — you compete for resources with live users and risk slowing down the app

Transitions: Report-oriented workloads optimize for read-time aggregation at scale. The next workload category prioritizes something different: correctness above all else.

Key Principles:

- Analytical workloads and transactional workloads have opposing optimization needs — serving both well usually means separating them
- Column-oriented storage is not better than row-oriented storage — it's optimized for a different access pattern

Examples:

- A retail company runs nightly reports on 3 years of sales data. Running that query on PostgreSQL locks tables and slows checkout. The same query on BigQuery completes in seconds and costs pennies.
- A SaaS product wants to show customers their usage trends. A daily ETL job pushes data to a warehouse; the dashboard reads from there, not from the production database.

02.2 Integrity-First Workloads 🕮

Learning Objectives:

- Understand what it means for a workload to require "integrity" and what breaks without it
- Identify which database features deliver integrity guarantees
- Know which databases are strongest when correctness is non-negotiable

Content Outline:

- What integrity means in database terms:
  - **Referential integrity**: foreign keys ensure that related records always exist (no orphaned orders pointing to deleted customers)
  - **Transactional integrity**: multi-step operations are atomic — either all succeed or none do
  - **Constraint integrity**: field-level rules enforced at write time (NOT NULL, UNIQUE, CHECK)
- Workloads that require integrity:
  - Financial transactions (debits and credits must balance)
  - Inventory management (stock levels must reflect actual reality)
  - Booking and reservation systems (double-booking must be prevented)
  - Order fulfillment (order, payment, and inventory must update together or not at all)
- How relational databases deliver integrity:
  - Foreign key constraints catch invalid references before writes commit
  - Transactions with ROLLBACK undo partial writes
  - Database-level constraints reject invalid data at the door
- NoSQL and integrity:
  - Most NoSQL databases offer document-level atomicity but not multi-document transactions
  - MongoDB added multi-document transactions in v4.0, but they come with performance costs
  - DynamoDB offers transactions but within strict limits
- When integrity is non-negotiable: use PostgreSQL, MySQL, or MS SQL Server — databases built from the ground up around ACID
- Anti-pattern: implementing integrity in application code instead of the database — code can have bugs, race conditions, and gaps; database constraints cannot be bypassed by any

application path

Transitions: Integrity-first workloads need writes that are always correct. The next lesson covers the opposite extreme: workloads dominated by reads, where speed and concurrency matter more than write guarantees.

Key Principles:

- Integrity is a database responsibility, not an application responsibility — push constraints as close to the data as possible
- "We'll validate it in the API" is not a substitute for database constraints — a second API, a direct DB access, or a migration can all bypass application-level validation

Examples:

- A hotel booking system: when a room is reserved, the inventory count must decrement and the booking record must be created in the same atomic transaction. A half-committed state leaves the room double-bookable.
- A payroll system: a salary payment creates a ledger entry, decrements a budget, and marks the payment as sent. All three must succeed together.

02.3 Query-Heavy Workloads 🕮

Learning Objectives:

- Understand what makes a workload "query-heavy" and why standard databases struggle at scale
- Identify strategies for handling high read volumes
- Know which databases and patterns are purpose-built for read-heavy access

Content Outline:

- What query-heavy means:
  - Far more reads than writes (e.g., 95% reads, 5% writes)
  - Many concurrent users reading simultaneously
  - Reads need to be fast — milliseconds, not seconds
- Why standard databases struggle under read load:
  - Every read acquires a shared lock, competing with other reads and writes
  - A single database node can only serve so many concurrent connections
  - Reads from disk are slow; reads from memory are fast
- Strategies for query-heavy workloads:
  1. **Read replicas**: copies of the primary database that serve read traffic; writes go to the primary, reads are distributed
  2. **Caching layers** (Redis, Memcached): frequently-accessed data stored in memory for sub-millisecond reads; the most requested data never hits the database at all
  3. **Column-oriented stores**: if the reads are aggregations over large datasets, a columnar database outperforms a row-oriented one
  4. **Denormalization**: structuring data for the specific query pattern rather than for normalization — reads become simpler, writes become more expensive

- When to add Redis:
  - Session data (user is logged in, their cart contents)
  - Frequently-read config or reference data that rarely changes
  - Leaderboards, counters, pub/sub messaging
  - The "first" million users can be served from cache; the database never sees them
- Anti-pattern: solving a read-heavy problem by upgrading server hardware (vertical scaling) instead of distributing the load — you hit a ceiling and the cost grows exponentially

Transitions: You've now seen how workload requirements shape the choice: reports call for columnar storage, integrity demands relational ACID, reads at scale need caching and replication. The next lesson brings it all together with a comparative view of the major databases in each category.

Key Principles:

- The fastest read is one that never reaches the database — caching is the first tool for read-heavy systems
- Read replicas and caching are not mutually exclusive — most high-scale systems use both

Examples:

- A news website serving millions of readers: article content is cached in Redis with a 10-minute TTL; only cache misses hit the database
- An e-commerce product catalog: read replicas handle search and listing pages; the primary database only handles order writes and inventory updates

---

02.4 The Major Databases Compared 🕮

Learning Objectives:

- Identify the major databases by category and what each is optimized for
- Apply workload criteria to narrow down options for a given scenario
- Understand the trade-offs of the most widely used databases in production

Content Outline:

- Overview of the categories:
  - Relational (SQL): PostgreSQL, MySQL, Microsoft SQL Server
  - Document: MongoDB, DynamoDB (also key-value)
  - Key-value: Redis, DynamoDB
  - Columnar/Analytical: BigQuery, Redshift, Snowflake
- Comparative table:

Database	Type	Best for	Managed?	Scale model
PostgreSQL	Relational	Integrity, complex queries, general-purpose	Self-host or cloud (RDS, Supabase)	Vertical + read replicas
MySQL	Relational	Web apps, high-read workloads, replication	Self-host or cloud (RDS, PlanetScale)	Vertical + read replicas

Database	Type	Best for	Managed?	Scale model
MS SQL Server	Relational	Enterprise, Windows stacks, BI integration	Self-host or Azure SQL	Vertical + always-on replicas
MongoDB	Document	Variable schemas, nested objects, content	MongoDB Atlas	Horizontal sharding
DynamoDB	Document / Key-value	High-scale serverless apps, AWS-native	Fully managed (AWS)	Horizontal, automatic
Redis	Key-value	Caching, sessions, leaderboards, pub/sub	Self-host or Upstash/Redis Cloud	Cluster horizontal
BigQuery	Columnar / Analytical	Analytics at scale, data warehousing	Fully managed (GCP)	Serverless, auto-scale

- How to use this table:
  - Step 1: identify your primary workload type (transactional, analytical, cache, document)
  - Step 2: apply your constraints (cloud provider, managed vs self-hosted, cost model)
  - Step 3: default to the simplest option that meets your requirements — optimize later
- Common pairings in production:
  - PostgreSQL (primary) + Redis (cache) + BigQuery (analytics)
  - DynamoDB (primary) + ElastiCache (cache) + Redshift (analytics)
- Anti-pattern: choosing based on name recognition or personal preference rather than workload fit — every database in this table has a domain where it wins and domains where it loses

Transitions: You've completed the decision framework. The assessment will ask you to apply it to real scenarios — choosing a database and justifying the choice from workload requirements.

Key Principles:

- Every database in this table is the best choice for something and the wrong choice for something else
- Production systems often combine multiple databases — one per access pattern
- "Start simple" is valid advice: PostgreSQL can handle a lot; add specialized databases when you have evidence that they're needed

Examples:

- A startup's first app: PostgreSQL for everything — it handles relational data, JSON documents via JSONB, and basic analytics. No need for complexity yet.
- At scale: the same app adds Redis for sessions and caching, BigQuery for reporting, and considers DynamoDB only if the write throughput outgrows PostgreSQL.

03.0 Database Selection Assessment 🧠

Learning Objectives:

- Apply the SQL vs NoSQL decision framework to concrete scenarios
- Match workload requirements (reporting, integrity, read-heavy) to the appropriate database
- Identify which major database fits a given use case and explain why

Question Format: Scenario-based multiple choice

Questions:

Q1. A food delivery company needs to store orders. Each order must link to a customer, a restaurant, and multiple items. If a payment fails mid-order, everything must roll back. Which database type fits best?

- A) Document database (MongoDB)
- B) Key-value store (Redis)
- C) Relational database (PostgreSQL) ✓
- D) Columnar database (BigQuery)

Why C: Multi-entity relationships and atomic rollback on payment failure require ACID guarantees. Relational databases are purpose-built for this.

Q2. A media company wants to analyze 5 years of video viewing data — total views per title per day, average watch time, geographic breakdowns. Queries run on hundreds of millions of rows. Which database type fits best?

- A) Relational database (PostgreSQL)
- B) Document database (MongoDB)
- C) Key-value store (Redis)
- D) Columnar data warehouse (BigQuery) ✓

Why D: Analytical queries on hundreds of millions of rows need columnar storage optimized for scan-and-aggregate, not row-by-row OLTP reads.

Q3. An e-commerce site's product catalog has clothing (size, material, color), electronics (wattage, connectivity, warranty), and furniture (dimensions, weight, material) — all with different attributes. Which database type fits best?

- A) Relational database with JSON columns
- B) Document database (MongoDB) ✓
- C) Key-value store (Redis)
- D) Columnar database (BigQuery)

Why B: Variable schemas per product type are a natural fit for document storage, where each document can have different fields without schema migration.

Q4. A social network serves 10 million users. 95% of traffic is reading profile pages and feeds. The database is struggling under read load. What is the most appropriate first response?

- A) Migrate from PostgreSQL to MongoDB
- B) Move the database to a larger server
- C) Add a Redis caching layer for frequently-read data ✓

- D) Split the database into multiple OLAP instances

Why C: Read-heavy load is best addressed first by caching — the fastest read is one that never reaches the database. Vertical scaling is expensive and has a ceiling.

Q5. A developer says: "We'll store financial transaction data in MongoDB because JSON is more flexible." What is the most significant risk?

- A) MongoDB is too expensive for financial data
- B) JSON cannot represent monetary values
- C) MongoDB's document model doesn't enforce multi-document transactional integrity by default ✓
- D) MongoDB doesn't support indexing

Why C: Financial transactions require atomic multi-step operations. MongoDB's default behavior doesn't guarantee cross-document ACID, and implementing it requires careful configuration with performance costs.

Q6. A gaming leaderboard needs to rank 1 million players and update scores thousands of times per second. Reads need to return the top 100 players in under 1 millisecond. Which database fits best?

- A) PostgreSQL with an index on the score column
- B) MongoDB with a sorted query
- C) BigQuery with a ranking query
- D) Redis with a sorted set ✓

Why D: Redis sorted sets are purpose-built for exactly this use case — O(log N) updates and O(log N + M) range queries at sub-millisecond speed from memory.

Q7. An enterprise app is considering adding an analytical dashboard for management reports. The production database is PostgreSQL and serves live users. What is the recommended approach?

- A) Run analytical queries directly on the production PostgreSQL instance
- B) Add read replicas to PostgreSQL and run analytics there
- C) Build an ETL pipeline to a data warehouse and run analytics there ✓
- D) Migrate the production database to BigQuery

Why C: Analytical queries compete with live traffic and should be isolated. An ETL pipeline to a warehouse keeps production performance unaffected and gives analytics the right storage model.

Evaluation Criteria:

- 7/7: Complete mastery of the decision framework
- 5-6/7: Strong understanding with minor gaps
- 3-4/7: Foundational understanding; review workload-matching sections
- Below 3/7: Review the full course from Section 01

Score Interpretation: Students who score below 5/7 should revisit the section that covers the workload type they missed. Each question maps to a specific lesson: Q1 → 02.2, Q2 → 02.1, Q3 → 01.2, Q4 → 02.3, Q5 → 01.1, Q6 → 02.3, Q7 → 02.1.

04.0 Conclusion

Content Outline:

- Course recap: what students accomplished
  - Built a mental model for the SQL vs NoSQL divide: what each model optimizes for and when each breaks down
  - Learned to match workloads to databases: report-oriented → columnar, integrity-first → relational, read-heavy → caching + replicas
  - Surveyed the major databases in production: PostgreSQL, MySQL, MS SQL Server, MongoDB, DynamoDB, Redis, BigQuery
- Connection to bigger picture:
  - Database selection is one layer of a larger architectural decision — your storage choice shapes your data pipeline design, your API performance, and your cost model
  - The next courses will cover how to collect and process telemetry — the workloads you just learned to route will be the pipelines that feed those systems
- Next steps and continued learning:
  - Hands-on: set up a PostgreSQL instance and a Redis cache for a small project; observe the latency difference between cached and uncached reads
  - Explore: read the official "Use Cases" pages for BigQuery, MongoDB, and DynamoDB — each vendor explains exactly what their database is built for
  - Go deeper: "Designing Data-Intensive Applications" by Martin Kleppmann is the canonical book on storage systems for software engineers
- Encouragement:
  - You now make storage decisions deliberately. Most developers don't — they pick what they know. You know *why* you're choosing, and that's a significant engineering advantage.

Note: This is conclusion only — no theory, exercises, or assessment.